Optimus OS — Software Design Document (SDD) v1.0

Oct 9, 2026 · @Eduardo

1. Introdução

Este SDD congela a arquitetura do Optimus OS v1 e libera o início da codificação pelo marco M0. Todas as decisões abertas nas conversas de planejamento foram tomadas aqui; mudanças posteriores entram como nova ADR (seção 3).

Propósito. O Optimus OS é um sistema operacional escrito do zero em Rust, em que o usuário conversa com uma IA que se apresenta como o próprio sistema. Por baixo, um microkernel pequeno e determinístico garante segurança e desempenho.

Escopo da v1.0

- Plataformas: x86-64 (UEFI) e AArch64 (UEFI/ACPI); RISC-V fica para depois da v1.0.
- Perfis: Lite (1 núcleo, 512 MB), Básico, JARVIS, Premium e Server.
- Inclui: kernel, serviços de sistema, drivers essenciais, OptFS, rede, compositor, Cockpit UI, AI Core com voz, apps em WebAssembly, atualizações seguras.
- Fora da v1.0: navegador web completo, hardware de mainframe (s390x), AR/VR, scripts complexos de texto (árabe, índico).

Público. Desenvolvedores do kernel, dos serviços, do AI Core e da interface; revisores de segurança.

Como ler. As seções 2 e 3 explicam o porquê. As seções 4 a 14 especificam o quê. As seções 15 a 20 dizem como construir e em que ordem. O backlog da seção 20 é o ponto de partida do código.

Convenções. "DEVE" é obrigatório, "DEVERIA" é recomendado, "PODE" é opcional. Nomes de crates e serviços aparecem em `monoespaçado`. Tamanhos usam KiB, MiB e GiB.

Realidade de escopo. Projetos comparáveis (Fuchsia, Redox, SerenityOS) levaram anos com equipes dedicadas. Por isso o roadmap entrega valor cedo: no marco M8 já existe um Optimus que conversa e age, ainda que só em máquina virtual.

2. Visão, princípios e requisitos

O Optimus é um assistente de IA que é o sistema operacional: o usuário fala, o sistema entende, planeja e age, com segurança verificável. Ele roda do perfil Lite (512 MB) ao servidor com dezenas de núcleos, com a mesma base de código.

2.1 Princípios de projeto (em ordem de prioridade)

1. **Segurança primeiro.** Nenhum componente tem poder total, nem a IA. Toda autoridade é uma capability explícita, mínima e revogável.
2. **Kernel determinístico.** O kernel não contém IA, não faz parsing de formatos complexos e cabe em 1 MiB.
3. **IA como interface única.** Voz, texto e HUD são a porta de entrada; apps e configurações são ferramentas da IA.
4. **Local por padrão.** Modelos, memória e dados ficam na máquina. A nuvem é opcional, visível e desligada de fábrica.
5. **Leveza mensurável.** Cada componente tem orçamento de RAM, CPU e latência (seção 15), verificado no CI.
6. **Tudo do zero, com padrões abertos.** Código próprio, formatos e protocolos públicos (UEFI, ACPI, virtio, WASI, GGUF, TLS 1.3).
7. **Falha contida e recuperável.** Serviço que cai é reiniciado; atualização ruim volta sozinha; existe modo de resgate sem IA.

2.2 Requisitos funcionais

ID	Requisito
RF-01	Dar boot via UEFI em x86-64 e AArch64, com Secure Boot e boot medido.
RF-02	Executar processos isolados com memória virtual, threads e IPC por capabilities.
RF-03	Suportar 1 a 256 núcleos lógicos e NUMA.
RF-04	Armazenar dados no OptFS com criptografia, snapshots e verificação de integridade.
RF-05	Conectar-se por Ethernet e Wi-Fi com IPv4/IPv6, TCP, UDP, DNS seguro e TLS 1.3.
RF-06	Exibir o Cockpit UI com aceleração por GPU ou renderização por software.
RF-07	Conversar por texto e voz em pt-BR e en-US, com interrupção (barge-in).
RF-08	Planejar e executar tarefas com ferramentas do sistema, sob política e confirmação.

ID	Requisito
RF-09	Manter memória de longo prazo local, criptografada e editável pelo usuário.
RF-10	Propor e aplicar melhorias de configuração e habilidades, com teste e aprovação.
RF-11	Instalar apps em WebAssembly isolados, com permissões declaradas.
RF-12	Atualizar o sistema de forma atômica (A/B), assinada e com rollback.
RF-13	Oferecer modo de resgate determinístico, sem IA.
RF-14	Operar sem monitor (headless) no perfil Server, com Cockpit remoto.
RF-15	Hospedar máquinas virtuais e realms (contêineres nativos) no perfil Server.

2.3 Requisitos não funcionais

ID	Categoria	Requisito
RNF-01	Segurança	Zero processos com privilégio total; superfície exposta à rede mínima e auditada.
RNF-02	Segurança	Resistir a prompt injection: conteúdo externo nunca dispara ação R3 sem confirmação física.
RNF-03	Desempenho	IPC ida e volta ⩽ 1 µs; troca de contexto ⩽ 2 µs (x86-64 moderno).
RNF-04	Leveza	Kernel ⩽ 1 MiB; RAM em repouso ⩽ 64 MiB no Lite.
RNF-05	Responsividade	Voz ponta a ponta ⩽ 800 ms no perfil JARVIS.
RNF-06	Confiabilidade	Serviço reiniciado em ⩽ 100 ms após falha; rollback de atualização em ⩽ 1 boot.
RNF-07	Privacidade	Nenhum dado sai da máquina sem política explícita; telemetria só local.
RNF-08	Portabilidade	Código dependente de arquitetura isolado em `arch/`; ⩾ 95% do kernel é portável.

ID	Categoria	Requisito
RNF-09	Manutenibilidade	`unsafe` só em módulos listados; cada bloco justificado e revisado por duas pessoas.
RNF-10	Acessibilidade	Todo fluxo operável só por voz, só por teclado ou com leitor de tela.

3. Decisões de arquitetura (ADRs)

As 24 decisões abaixo estão aceitas e valem para a v1.0. Cada uma vira um arquivo em `docs/adr/NNNN-titulo.md` no M0, com contexto, alternativas e consequências completas.

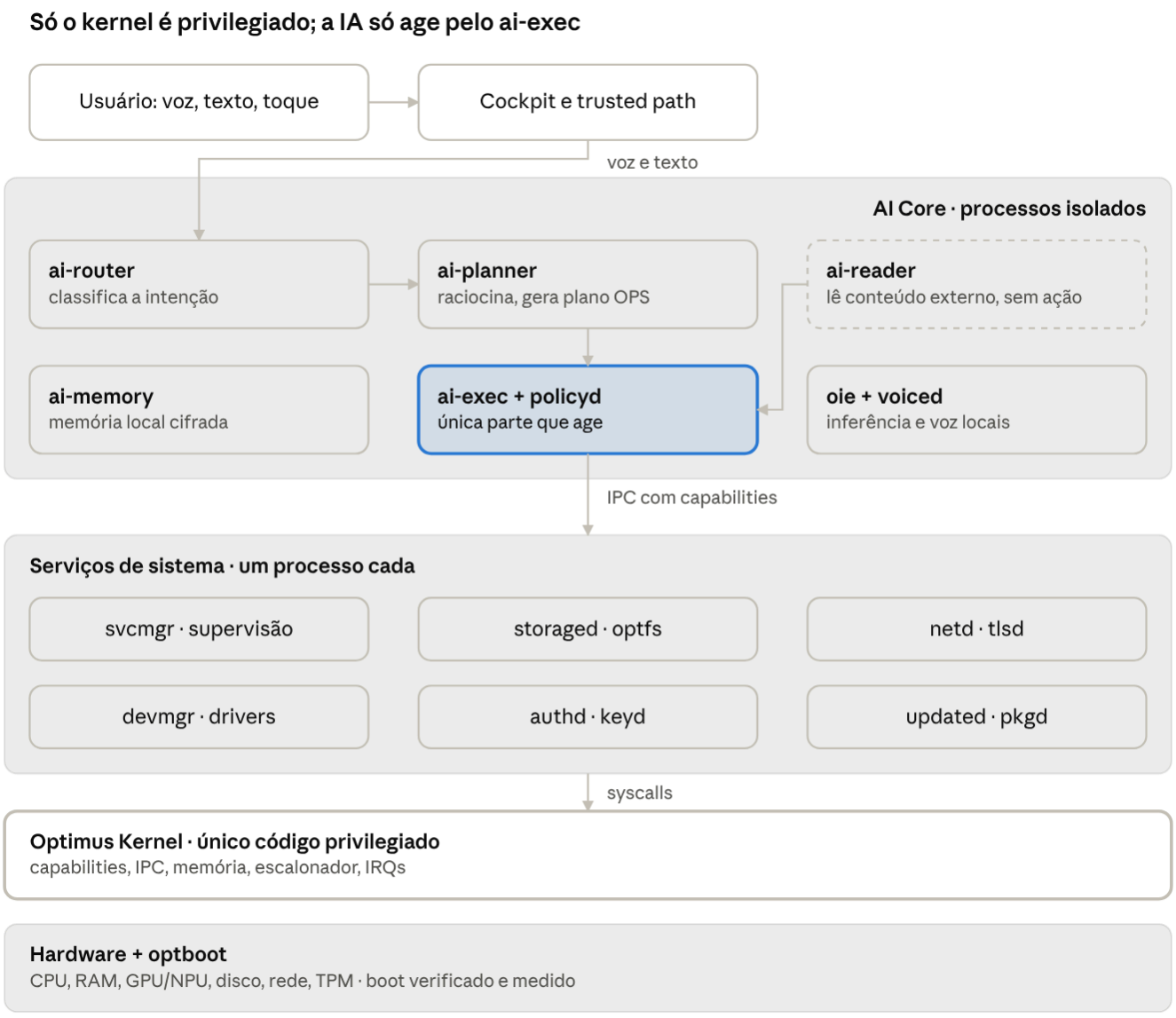
ADR	Decisão	Motivo	Consequência
0001	Microkernel com capabilities	Menor base de confiança; drivers isolados	Mais IPC; exige IPC muito rápido
0002	Rust estável (edition 2024) em todo o sistema; assembly só em `arch/`	Elimina classes de falhas de memória	Equipe precisa dominar Rust
0003	Tudo do zero; nenhuma crate externa na imagem do sistema	Controle total e auditoria	Mais código a escrever; dependências só em ferramentas de build e teste
0004	Exceção: primitivas criptográficas vêm de código gerado e formalmente verificado (estilo fiat-crypto/HACL*), adaptado na `optcrypt`	Cripto caseira é a maior fonte de falhas sutis	Única porta para código não escrito à mão; revisão dedicada
0005	Bootloader próprio `optboot` como aplicação UEFI; sem BIOS legado	UEFI dá Secure Boot, GOP e mapa de memória padronizado	Máquinas anteriores a ~2012 sem UEFI não são suportadas
0006	Alocação de objetos do kernel cobrada de capabilities `MemoryQuota`	Determinismo e resistência a DoS, mais simples que *untyped* do seL4	Todo serviço declara orçamento de memória

ADR	Decisão	Motivo	Consequência
0007	IPC síncrono (call/reply) + notificações assíncronas + anéis em memória compartilhada	Rápido para chamadas curtas, eficiente para volume	Três mecanismos a documentar e testar
0008	Escalonador de prioridade fixa (256 níveis) com *scheduling contexts* (orçamento/período)	Tempo real para áudio e UI; isolamento temporal	Política de prioridades centralizada (seção 6.6)
0009	Kernel *tickless*, não preemptível, com pontos de preempção em operações longas	Simplicidade e latência previsível	Operações longas DEVEM ser fatiadas
0010	SMP com filas por CPU e locks finos desde o projeto; v1 do M1 é single-core	Escala até servidores	Ordem de locks documentada e verificada
0011	Sem root e sem namespace global; cada processo recebe capabilities de diretório	Elimina escalada de privilégio clássica	Compatibilidade POSIX só via camada própria
0012	IDL própria (OIDL) gera stubs Rust e esquemas de ferramentas da IA	Uma fonte de verdade para IPC e para a IA	Compilador `oidlc` é peça central
0013	Drivers em espaço de usuário, com IOMMU obrigatória quando presente	Driver comprometido não domina a máquina	Máquinas sem IOMMU rodam em modo degradado e avisado
0014	OptFS: copy-on-write, criptografia autenticada por bloco, snapshots	Integridade, rollback e privacidade	FS próprio é um marco grande (M5)
0015	Imagem de sistema imutável, verificada por árvore de Merkle, em slots A/B	Sistema não pode ser alterado em repouso	Mudanças de sistema só por atualização
0016	Apps de terceiros em WebAssembly (Component Model/WASI), runtime próprio `owr`	Sandbox forte e portabilidade entre arquiteturas	Apps nativos só para o sistema

ADR	Decisão	Motivo	Consequência
0017	Pilha de rede própria em espaço de usuário, TLS 1.3 apenas, troca de chaves híbrida X25519+ML-KEM	Segurança moderna e pós-quântica	Sem compatibilidade com TLS 1.2
0018	Assinaturas híbridas Ed25519 + ML-DSA-65 em boot, pacotes e atualizações	Resistência a computação quântica	Assinaturas maiores (~3,3 KiB)
0019	IA fora do kernel, dividida em Planejador, Leitor isolado e Executor determinístico	Mitiga prompt injection (padrão inspirado no CaMeL)	IA mais lenta em tarefas sobre conteúdo externo
0020	Planos da IA em linguagem própria, limitada e não Turing-completa (OPS)	Planos verificáveis e com rastreio de origem dos dados	Planejador precisa gerar OPS com gramática
0021	Motor de inferência próprio `oie`, modelos em GGUF, quantização 4/8 bits, decodificação com gramática	Leveza e controle; formato aberto	Backend de GPU é trabalho grande (M12)
0022	Modelos só de pesos abertos com licença permissiva (Apache-2.0 ou MIT), escolhidos por avaliação na época do release	Distribuição legal e auditável	Lista de modelos revisada a cada release
0023	Interface gerada pela IA é declarativa (HUD Spec), nunca código	IA não executa código no compositor	Catálogo de componentes de UI finito
0024	Licença Apache-2.0 para o código do projeto	Concessão de patentes e adoção ampla	Contribuições exigem DCO

4. Arquitetura geral

O sistema tem quatro camadas: kernel, serviços de sistema, AI Core e Cockpit. Só o kernel roda em modo privilegiado; todo o resto são processos isolados que conversam por IPC com capabilities.



arquitetura · 4 camadas, fluxo de um pedido

O leitor (tracejado) lê conteúdo externo mas não tem nenhuma capability de ação; tudo o que muda o sistema passa pelo `ai-exec` e pela política.

4.1 Componentes

Camada	Componente	Papel
Firmware	`optboot`	Carrega e verifica kernel e bootfs; mede tudo no TPM
Kernel	`okernel`	Memória, threads, escalonamento, IPC, capabilities, IRQs, timers

Camada	Componente	Papel
Base	`root`	Primeiro processo; recebe todas as capabilities iniciais e inicia o `svcmgr`
Base	`svcmgr`	Supervisiona serviços, aplica manifestos e reinicia em falhas
Base	`memd`, `timed`, `logd`, `auditd`, `crashd`	Pressão de memória, relógio, logs, auditoria encadeada, relatórios de falha
Dispositivos	`devmgr` + drivers	Enumera ACPI/PCIe/DT e inicia um processo por driver
Armazenamento	`storaged`, `optfs`	Blocos, partições, sistema de arquivos
Rede	`netd`, `tlsd`	TCP/IP, firewall, DNS seguro, TLS
Interface	`displayd`, `inputd`, `audiod`	Compositor e trusted path, entrada, áudio
Segurança	`authd`, `keyd`, `policyd`	Usuários, chaves (TPM), políticas
Sistema	`updated`, `pkgd`, `owr`	Atualizações A/B, pacotes, runtime Wasm
IA	`ai-router`, `ai-planner`, `ai-reader`, `ai-exec`, `ai-memory`, `oie`, `voiced`, `evolved`	O AI Core (seção 12)
UI	`cockpit`	Shell da interface, renderiza HUD Spec
Resgate	`rescue`	Shell determinístico, sem IA

4.2 Fluxo principal de um pedido

1. O usuário fala; `audiod` entrega o áudio ao `voiced`, que transcreve.
2. `ai-router` classifica a intenção. Comandos simples vão direto ao `ai-exec`.
3. Pedidos complexos vão ao `ai-planner`, que gera um plano em OPS.
4. Dados externos citados no plano são processados pelo `ai-reader`, sem poder de ação.
5. `ai-exec` valida o plano contra a política (`policyd`), pede confirmação no trusted path se preciso e chama os serviços por IPC.

6. O resultado volta ao `cockpit` como HUD Spec e ao `voiced` como fala.

4.3 Regras de comunicação

- Toda chamada entre processos DEVE passar por uma interface OIDL.
- Nenhum serviço recebe capability que não esteja no seu manifesto assinado.
- Serviços de IA DEVEM ter as menores cotas de privilégio do sistema: o planejador e o leitor não têm nenhuma capability de ação.

5. Boot e cadeia de confiança

O `optboot` é uma aplicação UEFI em Rust que verifica, mede e carrega o kernel e o bootfs, e entrega ao kernel uma estrutura `BootInfo` versionada. Nenhum código não assinado executa entre o firmware e o primeiro processo.

5.1 Sequência

1. Firmware UEFI com Secure Boot valida a assinatura do `optboot` (chave própria do projeto inscrita no `db`, ou shim no futuro).
2. `optboot` lê `/EFI/optimus/slot-{a,b}/` conforme o slot ativo gravado em variável UEFI e no TPM.
3. Verifica as assinaturas híbridas (Ed25519 + ML-DSA-65) do kernel ELF e do bootfs.
4. Mede kernel, bootfs e linha de comando no PCR 8 e PCR 9 do TPM 2.0.
5. Coleta mapa de memória, framebuffer (GOP), RSDP da ACPI ou DTB e 64 bytes de entropia (EFI_RNG_PROTOCOL + RDSEED).
6. Escolhe o deslocamento de KASLR, monta as tabelas de página iniciais e chama `ExitBootServices`.
7. Salta para `kernel_entry(&BootInfo)` com interrupções desabilitadas.

5.2 `BootInfo` (v1)

Campo	Tipo	Conteúdo
`magic` / `version`	u64 / u32	`0x4F50_5449_4D55_5321` ("OPTIMUS!") / 1
`memory_map`	slice de `MemRegion`	Faixas normalizadas: usável, reservada, ACPI, MMIO, bootfs, kernel
`framebuffer`	Option	Endereço, largura, altura, stride, formato de pixel

Campo	Tipo	Conteúdo
`acpi_rsdp` / `dtb`	Option<u64>	Raiz das tabelas de firmware
`bootfs`	região	Imagem com `root`, `svcmgr`, drivers essenciais e manifestos
`cmdline`	str ⩽ 1 KiB	Opções do kernel (`log=`, `profile=`, `rescue`)
`rng_seed`	[u8; 64]	Semente inicial do CSPRNG
`kaslr_slide`	u64	Deslocamento aplicado
`tpm_event_log`	Option	Log de medições
`boot_time_ns`	u64	Carimbo para métricas de boot

5.3 bootfs

Formato próprio `OBFS`: cabeçalho, tabela de entradas (nome, deslocamento, tamanho, hash BLAKE3) e dados alinhados a 4 KiB, mapeáveis sem cópia. Contém apenas o necessário para montar a imagem de sistema verificada: `root`, `svcmgr`, `devmgr`, drivers de armazenamento, `storaged`, `optfs`, `keyd` e o `rescue`.

5.4 Chaves de disco e máquinas sem TPM

Com TPM, a chave do volume de dados é selada aos PCRs 0, 7, 8 e 9 e liberada só se o boot for o esperado. Sem TPM (comum em máquinas Lite antigas), a chave deriva de senha via Argon2id com parâmetros calibrados ao perfil (no Lite, 64 MiB e 3 iterações).

5.5 Falha de boot

Se o slot ativo falhar três vezes sem marcar sucesso, o `optboot` troca para o outro slot. Se ambos falharem, inicia o `rescue` a partir do bootfs.

6. Optimus Kernel (`okernel`)

O kernel oferece só sete serviços: objetos, capabilities, memória virtual, threads, escalonamento, IPC e eventos de hardware. A meta é ⩽ 25 mil linhas de Rust portável mais ⩽ 3 mil linhas de código de arquitetura, com imagem ⩽ 1 MiB.

6.1 Objetos do kernel

Objeto	Função	Direitos relevantes
`MemoryQuota`	Orçamento de bytes para criar objetos	`CREATE`, `SPLIT`
`AddressSpace`	Tabelas de página de um processo	`MAP`, `UNMAP`
`CapSpace`	Tabela de capabilities de um processo	`MANAGE`
`Thread`	Contexto de execução	`START`, `SUSPEND`, `READ_STATE`, `WRITE_STATE`
`SchedContext`	Orçamento de CPU (orçamento/período) e prioridade	`BIND`, `CONFIGURE`
`MemoryObject`	Conjunto de páginas físicas, opcionalmente contíguas ou de MMIO	`READ`, `WRITE`, `EXEC`, `MAP`
`Endpoint`	Ponto de IPC síncrono	`SEND`, `RECV`, `GRANT`
`Reply`	Capability de resposta de uso único	`SEND`
`Notification`	Palavra de 64 bits de sinais assíncronos	`SIGNAL`, `WAIT`
`Interrupt`	Linha de IRQ ou vetor MSI	`ACK`, `BIND`
`IoPortRange`	Portas de E/S (só x86)	`READ`, `WRITE`
`DmaDomain`	Domínio da IOMMU	`MAP`
`Vcpu` / `Vm`	Virtualização (M14)	`RUN`, `MAP`

Toda criação usa `obj_create(quota, tipo, args)` e debita bytes da `MemoryQuota`. Quando a última capability de um objeto é apagada, ele é destruído e os bytes voltam à cota.

6.2 Capabilities

- **Handle:** u32 = índice de 24 bits + geração de 8 bits, que evita reutilização acidental de slot.
- **CapSpace:** tabela radix de dois níveis com 4.096 slots por nível; cresce sob demanda, cobrada da cota.
- **Conteúdo do slot:** ponteiro para o objeto (contagem de referências), máscara de direitos de 32 bits e badge de 64 bits.
- **Derivação:** `cap_mint` cria uma cópia com direitos iguais ou menores e badge opcional. Direitos nunca aumentam.

- **Revogação:** cada capability guarda seus filhos numa árvore de derivação. `cap_revoke` apaga todos os descendentes, em fatias preemptíveis.
- **Transferência:** até 4 capabilities por mensagem de IPC, exigindo o direito `GRANT` no endpoint.

6.3 Memória física

- Alocador buddy de frames (ordens 0 a 18, 4 KiB a 1 GiB) por zona NUMA, com cache de frames por CPU.
- Alocador slab para objetos do kernel, com páginas de guarda nas pilhas do kernel (16 KiB por thread).
- O kernel não pagina a si mesmo. Toda memória do kernel é cobrada de alguma cota.

6.4 Memória virtual (x86-64, 48 bits)

Faixa	Uso
`0x0000_0000_0001_0000` a `0x0000_7FFF_FFFF_FFFF`	Espaço do usuário (primeiros 64 KiB nunca mapeados)
`0xFFFF_8000_0000_0000` (+ deslocamento aleatório)	Mapa direto da memória física, NX
`0xFFFF_C000_0000_0000`	Objetos e slabs do kernel
`0xFFFF_E000_0000_0000`	Áreas por CPU e pilhas do kernel
`0xFFFF_FFFF_8000_0000` (+ KASLR)	Imagem do kernel

Paginação de 4 níveis (5 níveis opcional no perfil Server). PCID para evitar flush de TLB; KPTI ativo só em CPUs vulneráveis a Meltdown. AArch64 usa TTBR0/TTBR1 com layout equivalente e ASIDs.

Faltas de página: o kernel não resolve faltas de usuário. Ele envia uma mensagem ao endpoint de exceções da thread; o `memd` ou o supervisor resolve, mapeia ou encerra o processo.

6.5 Endurecimento

- W^X estrito em kernel e usuário; SMEP, SMAP, UMIP e NX no x86; PAN, PXN, BTI e PAC no ARM; MTE quando disponível.
- Shadow stack (CET) em usuário e kernel quando suportado.
- KASLR e ASLR com 30+ bits de entropia em usuário.

- Pânico do kernel: congela as CPUs, grava o registro no log persistente e reinicia. Nunca continua em estado inconsistente.

6.6 Escalonador

- Prioridade fixa preemptiva, 256 níveis, round-robin dentro do nível (fatia padrão de 5 ms).
- Cada thread executa sobre um `SchedContext` com orçamento e período; sem orçamento, ela espera o próximo período.
- Doação de `SchedContext` em `ipc_call`: o servidor roda com o tempo do cliente, o que evita inversão de prioridade.
- Filas por CPU; balanceamento por migração periódica e em wake-up, respeitando afinidade e NUMA.

Faixa	Uso
240–255	Áudio, entrada, compositor (tempo real)
200–239	Drivers
128–199	Serviços de sistema e `ai-exec`
64–127	Apps e IA em primeiro plano
1–63	Fundo: indexação, raciocínio longo, `evolved`
0	Ocioso (mwait / WFI)

6.7 IPC

- **Mensagem:** rótulo de 64 bits + até 6 palavras em registradores (caminho rápido) ou até 512 palavras no buffer de IPC de 4 KiB por thread + até 4 capabilities.
- **Primitivas:** `ipc_call`, `ipc_reply_recv`, `ipc_send`, `ipc_recv`, todas com prazo opcional.
- **Caminho rápido:** call e reply entre threads na mesma CPU, mensagem só em registradores, troca direta de contexto sem passar pelo escalonador. Meta ≤ 1 µs ida e volta.
- **Volume:** dados grandes usam anéis SPSC em `MemoryObject` compartilhado com `Notification` para despertar.
- **Eventos:** qualquer fonte (IRQ, endpoint com mensagem pendente, timer) pode ser ligada a um bit de `Notification` via `obj_bind_notify`. O runtime assíncrono do usuário espera numa única notificação.

6.8 Interrupções, tempo e energia

- x86: IDT própria, x2APIC, MSI/MSI-X preferidos, IOAPIC como reserva. ARM: GICv3.
- IRQs de dispositivos viram sinais de `Notification` para o driver dono da capability `Interrupt`.
- Relógio monotônico em ns a partir do TSC invariante (calibrado contra HPET/ACPI PM) ou do timer genérico ARM.
- Timer por CPU em modo deadline, sem tick periódico.
- CPU ociosa entra em C-states via `mwait`; frequência controlada pelo serviço `powerd` com capabilities de MSR.

6.9 SMP

- Partida dos núcleos via ACPI MADT (INIT-SIPI-SIPI) ou PSCI no ARM.
- Estado do kernel por CPU; locks por objeto (ticket spinlock) com ordem fixa: CapSpace → objeto → AddressSpace → fila do escalonador.
- TLB shootdown por IPI, agrupado e limitado às CPUs que usaram o AddressSpace.
- Futex próprio (`futex_wait` / `futex_wake`) indexado por (AddressSpace, endereço) para mutexes de usuário.

7. Serviços de sistema

Todo serviço é um processo Rust `no_std` sobre a biblioteca `ostd`, declarado num manifesto assinado e supervisionado pelo `svcmgr`. Cada um tem cota de memória, prioridade e lista fechada de capabilities.

7.1 Runtime de usuário: `ostd` e `orun`

- `ostd`: alocador (estilo mimalloc, próprio), threads, `Mutex` / `Condvar` sobre futex, coleções, strings UTF-8, tempo, logs estruturados e clientes OIDL.
- `orun`: executor assíncrono por thread que espera numa `Notification`; tarefas `async` para IPC, timers e anéis.
- Não existe libc no sistema base. A `olibc` (M15) serve só à camada POSIX.

7.2 `root` e `svcmgr`

- `root` recebe do kernel todas as capabilities iniciais, divide a memória em cotas e inicia o `svcmgr`. Depois disso só responde ao `svcmgr`.
- `svcmgr` lê manifestos do bootfs e da imagem de sistema, resolve dependências (grafo acíclico verificado no build) e inicia serviços em paralelo.

- Política de reinício: `always`, `on-failure` ou `never`, com recuo exponencial (100 ms a 30 s) e limite de 5 falhas por minuto antes de alertar.
- Ao reiniciar, as capabilities do processo antigo são revogadas e reemitidas ao novo; clientes recebem `PeerClosed` e reconectam.

7.3 Serviços de base

Serviço	Responsabilidade
`memd`	Resolve faltas de página, memória comprimida (LZ4) sob pressão, mapeamento de arquivos
`timed`	Relógio de parede, fusos horários, sincronização por NTS (NTP com autenticação)
`logd`	Logs estruturados em anéis por serviço; persistência rotativa em `/data/log`
`auditd`	Log de auditoria somente-anexo, encadeado por hash BLAKE3 e assinado por época
`crashd`	Minidumps próprios, análise e relatório local
`powerd`	Frequência de CPU, suspensão, hibernação, perfis de energia
`authd`	Contas, login, sessões, FIDO2, biometria opcional
`keyd`	Chaves de disco e de usuário; único processo com acesso ao TPM
`policyd`	Avalia políticas de segurança e da IA (linguagem OPL, seção 12.6)
`updated`	Atualizações A/B, metadados no estilo TUF, deltas
`pkgd`	Instala e remove pacotes `.opk`, gerencia permissões de apps
`rescue`	Shell textual determinístico: estado, logs, rollback, rede, reset da IA

7.4 Namespaces e realms

Não existe árvore de arquivos global. Cada processo recebe capabilities de diretório para as pastas que precisa, montadas num namespace próprio (`/sys`, `/data/app/<id>`, `/tmp`). Um **realm** é um grupo de processos com namespace, cotas e política comuns;

apps, a IA e contêineres do Server são realms.

8. Drivers

Cada driver é um processo isolado, criado pelo `devmgr` com acesso só ao seu dispositivo: regiões MMIO, IRQs e um `DmaDomain` da IOMMU. Um driver que trava é reiniciado sem derrubar o sistema.

8.1 Optimus Driver Framework (ODF)

- **Enumeração:** `devmgr` lê ACPI (via `acpid`, que contém o interpretador AML próprio, isolado) ou Device Tree, varre PCIe e publica nós de dispositivo.
- **Associação:** cada driver tem um manifesto com regras de casamento (IDs PCI, classes, `compatible` do DT). O mais específico vence.
- **Capabilities entregues:** `MemoryObject` dos BARs, `Interrupt` (MSI-X), `DmaDomain`, `IoPortRange` quando necessário.
- **API:** traits Rust assíncronas (`BlockDevice`, `NetDevice`, `InputDevice`, `AudioDevice`, `DisplayDevice`) expostas via OIDL às camadas de cima.
- **Sem IOMMU:** o driver roda mesmo assim, mas o sistema marca a máquina como "isolamento de DMA ausente" no HUD e no `auditd`.
- **Firmware binário:** blobs de fabricante (Wi-Fi, GPU) só carregam se estiverem no catálogo assinado, com hash fixo, e rodam no dispositivo, nunca na CPU.

8.2 Drivers por marco

Marco	Drivers
M1	UART 16550, framebuffer GOP (no kernel, só para log de pânico)
M4	PCIe, IOMMU Intel VT-d e AMD-Vi, virtio (blk, net, input, gpu 2D, rng, console), PS/2, RTC, ACPI básico
M5	NVMe, AHCI/SATA
M9	Intel HDA, virtio-sound
M10	TPM 2.0 (CRB e TIS)
M12	xHCI, USB HID, USB mass storage, USB áudio, Intel e1000e/i219, Realtek RTL8125, Intel Wi-Fi (série AX/BE), GPU Intel Xe (display e depois compute), AArch64: GICv3, PL011, SMMUv3

Marco	Drivers
Pós-v1	GPU AMD RDNA, NPUs, Bluetooth, câmeras UVC, RISC-V

8.3 Regras para escrever drivers

- Todo parser de descritores do dispositivo (PCI, USB, ACPI) DEVE ter alvo de fuzzing.
- Nenhum driver acessa memória fora dos `MemoryObject` recebidos.
- Drivers DEVEM tolerar reinício: estado persistente fica no serviço de cima, não no driver.

9. Armazenamento e OptFS

O disco tem três áreas: partição EFI, imagem de sistema imutável em dois slots (A/B) e um volume OptFS de dados criptografado. O sistema nunca escreve na imagem de sistema em uso.

9.1 Layout do disco (GPT)

Partição	Tamanho típico	Conteúdo
ESP (FAT32)	256 MiB	`optboot`, kernels e bootfs dos slots A e B
`sys-a` / `sys-b`	1 GiB cada (Lite: 384 MiB)	Imagem de sistema somente leitura, verificada por Merkle
`data`	Restante	Volume OptFS: usuários, apps, modelos de IA, memória da IA, logs

9.2 Imagem de sistema verificada

A imagem é um OptFS somente leitura com árvore de Merkle SHA-256 sobre blocos de 4 KiB. A raiz da árvore está no manifesto assinado do slot. Cada leitura verifica o caminho até a raiz; bloco divergente gera erro de E/S e alerta, nunca dado corrompido.

9.3 OptFS (volume de dados)

- **Copy-on-write** com árvore B+ de extents; nada é sobrescrito no lugar, então um corte de energia nunca corrompe o volume.
- **Integridade:** cada ponteiro de bloco guarda o tag de autenticação do bloco filho, formando uma árvore autenticada até o superbloco.

- **Criptografia:** AES-256-GCM por bloco, ou ChaCha20-Poly1305 em CPUs sem AES-NI (comum no Lite). Nonce de 96 bits no ponteiro do bloco.
- **Chaves:** chave mestra do volume (via `keyd`), chave por usuário e chave por app, derivadas por HKDF-SHA-256.
- **Snapshots** instantâneos, usados antes de mudanças da IA e de atualizações.
- **Compressão** LZ4 por padrão; zstd nível 3 para arquivos frios (decodificadores próprios).
- **Recursos:** links simbólicos dentro do namespace, atributos estendidos, cotas por realm, TRIM.
- **Recuperação:** dois superblocos rotativos com contador de transação; `optfs-check` no modo de resgate.

9.4 Pilha de armazenamento

`driver NVMe/AHCI/virtio-blk` → `storaged` (filas, GPT, agendamento de E/S com prioridade herdada do cliente) → `optfs` (um processo por volume) → clientes por capability de diretório. Arquivos podem ser mapeados em memória via `MemoryObject` com páginas fornecidas pelo `memd`, o que permite ao `oie` carregar modelos sem cópia.

10. Rede

A rede é negada por padrão: nenhuma conexão de entrada é aceita e nenhum processo abre conexão de saída sem uma capability `NetAccess` que lista destinos permitidos. A pilha inteira roda em espaço de usuário, no `netd`.

10.1 Pilha `netd`

- Camada 2: Ethernet, ARP, VLAN (Server); Wi-Fi via supplicant próprio `wland` com WPA3-SAE e WPA2 apenas como fallback avisado.
- Camada 3: IPv4, IPv6, ICMP/ICMPv6, DHCPv4, DHCPv6, SLAAC com endereços temporários de privacidade.
- Camada 4: TCP (CUBIC padrão; BBR opcional no Server), UDP; QUIC no M14.
- DNS: resolvedor próprio com cache, DNS-over-HTTPS e DNS-over-TLS padrão, DNSSEC validado.
- Desempenho: anéis de pacotes sem cópia entre driver e `netd`; no Server, multi-fila com RSS e uma thread por fila.

10.2 Firewall

- Entrada: tudo bloqueado, salvo serviços listados no manifesto e aprovados pelo usuário.
- Saída: avaliada por processo contra a capability `NetAccess` (domínios, IPs, portas, protocolo).
- A IA não tem acesso direto à rede. Ela pede ao `ai-exec`, que usa ferramentas de rede (busca, download) com política e auditoria.
- Limites de taxa e de conexões por processo contra abuso e exfiltração.

10.3 TLS e cripto de rede

- `tlsd` implementa só TLS 1.3, com troca de chaves híbrida X25519+ML-KEM-768, AES-256-GCM e ChaCha20-Poly1305.
- Certificados X.509 com parser próprio e fuzzing contínuo; raízes confiáveis na imagem de sistema; verificação de revogação por OCSP stapling e CRLite.
- Chaves privadas de sessão nunca saem do processo `tlsd`.

10.4 Serviços de rede do sistema

- `updated` fala apenas com o servidor de atualização fixado (pinning de chave).
- Cockpit remoto (Server): Optimus Remote Protocol sobre QUIC com autenticação mútua por certificado + FIDO2.

11. Gráficos, Holo UI e Cockpit

O Cockpit é a cara do Optimus: um HUD escuro, centrado numa esfera que representa a IA, com painéis que a própria IA monta conforme o pedido. O design é original, inspirado em HUDs futuristas, sem reproduzir elementos de marcas ou filmes.

11.1 Pilha gráfica

Camada	Componente	Detalhe
Driver	`gpu-virtio`, `gpu-xe`, framebuffer	virtio-gpu 2D no M4; Intel Xe no M12
Abstração	`gpu-hal`	API explícita estilo Vulkan: buffers, filas, fences, shaders SPIR-V
Renderizador	`forge`	Vetores, texto e efeitos; backends GPU e software (SIMD)

Camada	Componente	Detalhe
Compositor	`displayd`	Composição, vsync, multi-monitor, HDR (pós-v1), trusted path
Toolkit	`holo`	UI declarativa em Rust, layout flexbox, animações, acessibilidade
Shell	`cockpit`	Interface do sistema e renderizador de HUD Spec

- Protocolo de janelas próprio: clientes entregam buffers em `MemoryObject` com fences; o compositor nunca lê memória do cliente fora deles.
- Texto: parser OpenType e rasterizador próprios, com hinting leve e cache de glifos na GPU; shaping de latim, grego e cirílico na v1.
- Fontes do sistema: famílias livres com licença OFL, empacotadas na imagem.

11.2 Trusted path

- Uma faixa no topo da tela (a "Faixa de Confiança") só pode ser desenhada pelo `displayd`.
- Confirmações de ações R3, pedidos de senha e alertas de segurança aparecem só nela.
- A combinação **Super+Esc** (sequência de atenção segura) é capturada pelo `inputd` e sempre leva à tela de bloqueio ou confirmação real.

11.3 Modos do Cockpit

Modo	Quando	Elementos
Boot	Inicialização	Animação curta dentro do orçamento de boot
Repouso	Sem tarefa ativa	Esfera, anéis de estado (CPU, RAM, rede, segurança), relógio, sugestões
Conversa	Usuário fala ou digita	Linha do tempo da conversa, transcrição ao vivo, esfera reativa à voz
Tarefa	Plano em execução	Cartões com passos, progresso, ações pendentes de confirmação
Foco	App em tela cheia	Mini-HUD recolhível
Alerta	Evento de segurança	Faixa de Confiança destacada, ações sugeridas
Resgate	Falha grave ou pedido	Terminal textual do `rescue`

11.4 HUD Spec

A IA descreve painéis em JSON validado por esquema, nunca em código. Componentes permitidos na v1: `text`, `markdown`, `table`, `list`, `chart` (linha, barras, área), `metric`, `progress`, `form`, `image`, `media`, `file-grid`, `timeline`, `map` (pós-v1). Dados vêm por tokens de capability de leitura, e ações em formulários viram novos pedidos ao `ai-exec`.

11.5 Orçamentos por perfil

- Lite: renderização por software, 2D sem desfoque, 30 fps, ⩽ 24 MiB para compositor + cockpit.
- Básico: software SIMD ou GPU simples, 30–60 fps.
- JARVIS/Premium: GPU, 60 fps em 1440p, desfoques e partículas, quadro ⩽ 8 ms.
- Acessibilidade: alto contraste, redução de movimento, escala de interface 100–300%, leitor de tela integrado à voz da IA.

12. AI Core

Para o usuário, o Optimus é uma única IA que conversa, raciocina e opera o computador. Por dentro, são processos separados por função e privilégio: quem lê conteúdo externo não age, e quem age não interpreta texto livre.

12.1 Processos

Processo	Faz	Capabilities
`ai-router`	Classifica a intenção em < 20 ms; resolve comandos simples por gramática	Envia ao `ai-exec` e ao `ai-planner`
`ai-planner`	Conversa, raciocina e gera planos OPS	Só `oie` e leitura da memória; nenhuma ação
`ai-reader`	Lê conteúdo externo e devolve dados estruturados por esquema	Só `oie`; sem rede, sem memória, sem ação
`ai-exec`	Interpreta OPS, aplica política, pede confirmação, chama ferramentas	Capabilities das ferramentas permitidas
`ai-memory`	Memória de longo prazo e busca vetorial	Volume `/data/ai`
`oie`	Inferência de modelos	GPU/NPU, arquivos de modelos
`voiced`	Palavra de ativação, VAD, STT, TTS	`audiod`

Processo	Faz	Capabilities
`evolved`	Autoaprimoramento (12.10)	Sandbox de testes, propostas ao `ai-exec`
`ctxd`	Contexto e sugestões proativas	Telemetria somente leitura, agenda

12.2 Ciclo de um pedido

1. **Entender:** `ai-router` decide entre caminho rápido ("abra os arquivos", "volume 50%") e planejamento.
2. **Planejar:** `ai-planner` recebe o pedido, o contexto confiável e a memória relevante, e gera um plano OPS com decodificação restrita por gramática.
3. **Verificar o plano:** o planejador faz uma autocrítica com orçamento limitado; `ai-exec` checa tipos, ferramentas e política antes de executar.
4. **Executar:** `ai-exec` roda o plano passo a passo. Conteúdo externo vai ao `ai-reader` e volta como variável marcada.
5. **Conferir:** o resultado é comparado com o objetivo; se falhar, o planejador replaneja até 3 vezes.
6. **Responder:** HUD Spec ao `cockpit` e fala ao `voiced`; tudo registrado no `auditd`.

12.3 Proveniência (taint)

Todo dado carrega uma etiqueta de origem: `USER` (entrada direta do usuário pelo trusted input), `SYSTEM`, `LOCAL(dono)`, `EXTERNAL` (rede, e-mail, web, arquivos baixados) ou `AI` (gerado por modelo). Etiquetas se propagam: um valor derivado de `EXTERNAL` continua `EXTERNAL`. O planejador nunca vê texto `EXTERNAL` cru, só resumos e campos do leitor, sempre marcados.

12.4 OPS (Optimus Plan Script)

Linguagem pequena, tipada e sem laços infinitos: chamadas de ferramenta, variáveis imutáveis, `if`, `for` sobre listas finitas e `ask_user`. O executor conhece a origem de cada variável.

```
plan "resumir e-mails de hoje" {
  let msgs = mail.list(folder: "inbox", since: today())
  let resumo = reader.summarize(msgs, schema: Resumo)   // EXTERNAL
  ui.show(panel: "lista", data: resumo)
  if resumo.urgentes.len() > 0 {
    ask_user("Quer responder aos urgentes agora?")
  }
}
```

12.5 Ferramentas e classes de risco

Ferramentas são métodos OIDL anotados com `@tool`. O `oidlc` gera o esquema que o planejador vê e a ligação que o executor chama.

Classe	Exemplos	Regra
R0	Ler status, listar arquivos, buscar na memória	Livre, auditado
R1	Abrir app, criar arquivo, ajustar volume	Livre, reversível
R2	Mover para lixeira, editar documento, baixar arquivo	Livre se origem `USER`; confirmação na tela se envolver `EXTERNAL`
R3	Enviar mensagem, pagar, apagar definitivamente, mudar segurança, instalar app	Sempre confirmação física no trusted path
R4	Mexer em kernel, chaves, cadeia de atualização, desligar auditoria	Proibido para a IA

12.6 Política (OPL)

O `policyd` avalia regras declarativas e determinísticas antes de cada passo. Regras de fábrica são assinadas; o usuário pode endurecê-las livremente, mas afrouxar exige o app Configurações no trusted path. A IA só pode propor endurecimentos.

```
rule "e-mail para fora exige confirmação" {
  when tool == "mail.send" and recipient.domain not in user.trusted_domains
  then confirm(level: physical, show: [recipient, subject, provenance])
}
rule "dado externo não escolhe destinatário" {
  when tool.risk >= R3 and arg("recipient").taint == EXTERNAL
  then confirm(level: physical, warn: "destinatário veio de conteúdo
externo")
}
```

12.7 Raciocínio

- Orçamento de computação por perfil: no Lite, uma passada; no JARVIS, até 4 planos candidatos com verificação; no Premium, busca em árvore limitada e verificação por um segundo modelo.
- Ferramentas de raciocínio: calculadora exata, execução de código em sandbox Wasm, busca na memória e na documentação local.
- Honestidade: o planejador DEVE dizer quando não sabe ou quando um passo falhou; o HUD mostra o grau de confiança.

12.8 Memória

Tipo	Conteúdo	Armazenamento
Episódica	Conversas e ações, com data e origem	Log criptografado em `/data/ai/episodes`
Semântica	Fatos e documentos indexados	Índice vetorial HNSW próprio + embeddings locais
Perfil	Preferências confirmadas pelo usuário	Registro estruturado
Procedimental	Habilidades: planos OPS que deram certo e foram aprovados	Biblioteca de skills versionada

O usuário pode perguntar "o que você sabe sobre mim?", ver, editar e apagar qualquer item. Memórias de origem `EXTERNAL` nunca viram instrução.

12.9 Voz

- Pipeline: palavra de ativação ("Optimus") → detecção de voz → STT em streaming → planejamento → TTS em streaming.

- Interrupção (barge-in) com cancelamento de eco próprio (filtro adaptativo NLMS + supressão espectral).
- Voz nunca autoriza R3. Verificação de locutor é só conveniência, não autenticação.
- pt-BR e en-US na v1; voz sintética própria, sem imitar pessoas reais.

12.10 Persona e contexto

- Persona padrão: cordial, objetiva, com humor leve; configurável em tom e voz. A persona nunca sobrepõe política.
- `ctxd` cruza agenda, atividade e telemetria (opt-in) para sugestões proativas, com limite de frequência e modo "não perturbe".

12.11 Autoaprimoramento (`evolved`)

1. Observa métricas, falhas e pedidos repetidos.
2. Gera uma proposta: ajuste de parâmetro, nova skill, nova ferramenta Wasm ou layout de painel.
3. Testa num realm isolado ou VM com snapshot, contra benchmarks e a suíte de segurança.
4. Mostra o relatório no Cockpit; aplica só com aprovação, como transação com snapshot.
5. Monitora e faz rollback automático se piorar.

O `evolved` NUNCA altera kernel, serviços da base de confiança, políticas (exceto endurecer), chaves ou a cadeia de atualização. Melhorias no código do próprio Optimus viram propostas de patch exportadas para os desenvolvedores, não aplicadas na máquina. Ajuste fino local de modelos (LoRA) fica para depois da v1, opt-in e só no Premium.

12.12 Motor de inferência `oie`

- Arquiteturas: transformers só-decodificador (RoPE, GQA, SwiGLU, RMSNorm), codificador-decodificador para STT e modelos de TTS compactos.
- Formatos: GGUF; quantização Q4_K, Q5_K, Q8_0, F16 e BF16.
- Pesos mapeados do disco sem cópia; cache KV paginado e quantizado em 8 bits; cache de prefixo para o prompt do sistema.
- Kernels SIMD: SSE2 (base), AVX2/FMA, AVX-512/VNNI, AMX, NEON, SVE. GPU Intel Xe no M12.
- Decodificação especulativa com modelo rascunho (JARVIS+); lotes contínuos no Server.
- Decodificação restrita por gramática (OPS, JSON Schema) para chamadas de ferramenta sempre válidas.

12.13 Modelos por perfil

Perfil	Roteador	Planejador	Leitor	Voz
Lite	Classificador ≤ 10 MiB	0,5–1B (Q4), sob demanda	Mesmo modelo, outra sessão	Comandos por palavra-chave; STT tiny
Básico	Classificador	3–8B (Q4)	1–3B	STT small, TTS compacto
JARVIS	Classificador	14–32B (Q4)	3–8B	STT medium em streaming
Premium	Classificador	70B (Q4) + verificador	8–14B	STT large

Modelos são pacotes `.omdl` assinados com licença, hash e notas de avaliação. A escolha final de cada modelo acontece no release, por benchmark (ADR 0022).

12.14 Nuvem opcional

O provedor remoto vem desligado. Ligado, recebe só o necessário, por TLS, com indicador permanente no HUD e registro no `auditd`. Dados `LOCAL` marcados como privados nunca saem.

13. Segurança e modelo de ameaças

A base de confiança (TCB) é só `optboot`, `okernel`, `root`, `svcmgr`, `keyd`, `policyd`, `displayd` (trusted path) e `updated`. Tudo o mais é considerado comprometível, e o sistema deve continuar seguro se qualquer outro processo for tomado.

13.1 Ativos

Dados pessoais e memória da IA; chaves e credenciais; integridade do kernel, da imagem e das atualizações; controle da IA (só o usuário a dirige); disponibilidade do sistema.

13.2 Ameaças e controles

Ameaça	Controles	Seções
Exploração remota de serviço de rede	Rede negada por padrão, `netd` isolado, parsers com fuzzing, Rust	10, 18

Ameaça	Controles	Seções
Falha de memória em código	Rust, `unsafe` mínimo e revisado, Miri, Kani	17, 18
Driver ou dispositivo malicioso	Drivers em usuário, IOMMU, catálogo de firmware	8
Prompt injection por conteúdo externo	Leitor isolado, taint, OPS, política, confirmação física	12
IA usada para exfiltrar dados	Sem rede direta, `NetAccess`, auditoria, limite de taxa	10, 12
Escalada de privilégio	Sem root, capabilities atenuáveis e revogáveis	6, 7
Atualização ou modelo adulterado	Assinatura híbrida, TUF, Merkle, A/B, contador anti-rollback	5, 9, 14
Cadeia de suprimentos do build	Builds reproduzíveis, zero crates na imagem, chaves offline	16
Roubo do dispositivo	Disco criptografado, chave selada no TPM, proteção de DMA	5, 9
Spoofing de interface	Trusted path exclusivo do `displayd`, Super+Esc	11
Negação de serviço	Cotas de memória, `SchedContext`, limites de IPC e rede	6, 10
Engenharia social	Confirmações mostram origem e risco em linguagem clara	11, 12

13.3 Criptografia

Uso	Algoritmo
Assinaturas	Ed25519 + ML-DSA-65 (híbrido)
Troca de chaves	X25519 + ML-KEM-768 (híbrido)
Cifra simétrica	AES-256-GCM; ChaCha20-Poly1305
Hash	SHA-256, SHA-384; BLAKE3 para endereçamento interno
Derivação	HKDF-SHA-256; Argon2id para senhas

Uso	Algoritmo
Aleatoriedade	CSPRNG do kernel (ChaCha20) semeado por RDSEED, TPM e jitter

Toda implementação passa nos vetores Wycheproof e NIST e em testes de tempo constante (dudect) no CI.

13.4 Chaves do projeto

Raiz offline em dois tokens de hardware separados, cerimônia documentada e gravada. Chaves de release online são intermediárias com validade de 1 ano, rotacionadas pela raiz no estilo TUF.

13.5 Riscos aceitos

Canais laterais de microarquitetura (mitigados parcialmente), hardware ou firmware comprometido de fábrica, adversários com recursos de Estado e máquinas sem IOMMU ou sem TPM (suportadas em modo degradado e avisado).

13.6 Processo

`SECURITY.md` com canal privado de reporte, correção em até 30 dias para falhas críticas, avisos públicos e CVEs. Revisão externa de segurança antes da v1.0.

14. Interfaces e formatos

O kernel expõe 34 syscalls estáveis; todo o resto é OIDL entre processos. Os formatos abaixo são a referência para o código do M0 em diante e ficam versionados na crate `kabi` e em `docs/spec/`.

14.1 Convenção de chamada

- **x86-64:** instrução `syscall`; número em `rax`; argumentos em `rdi`, `rsi`, `rdx`, `r10`, `r8`, `r9`; palavras de mensagem extras em `r12` – `r15`. Retorno: `rax` = status, `rdx` e seguintes = valores.
- **AArch64:** `svc #0`; número em `x8`; argumentos em `x0` – `x5`; mensagem em `x6` – `x7` e `x9` – `x12`.
- Status `0` = sucesso; negativo = erro de `kabi::Error`.

14.2 Tabela de syscalls (ABI v1)

Nº	Nome	Nº	Nome
0	`debug_write` (só com cap de debug)	17	`sched_bind`
1	`cap_mint`	18	`ipc_send`
2	`cap_delete`	19	`ipc_recv`
3	`cap_revoke`	20	`ipc_call`
4	`cap_info`	21	`ipc_reply_recv`
5	`obj_create`	22	`ntfy_signal`
6	`vm_map`	23	`ntfy_wait`
7	`vm_unmap`	24	`ntfy_poll`
8	`vm_protect`	25	`obj_bind_notify`
9	`mem_op` (commit, zero, cache)	26	`irq_ack`
10	`thread_configure`	27	`clock_get`
11	`thread_start`	28	`yield`
12	`thread_suspend`	29	`futex_wait`
13	`thread_resume`	30	`futex_wake`
14	`thread_read_state`	31	`dma_map` / `dma_unmap` (via flag)
15	`thread_write_state`	32	`sys_info`
16	`sched_configure`	33	`thread_exit`

Erros: `InvalidHandle`, `WrongType`, `AccessDenied`, `QuotaExceeded`, `InvalidArgument`, `WouldBlock`, `TimedOut`, `PeerClosed`, `Cancelled`, `NotSupported`, `Busy`, `Revoked`.

14.3 OIDL

Arquivos `.oidl` em `idl/` geram, pelo `oidlc`, stubs Rust de cliente e servidor, serialização sem cópia (layout fixo, little-endian) e esquemas de ferramentas da IA.

```
package optimus.fs @1.0;

struct DirEntry { name: string<255>, kind: EntryKind, size: u64, modified:
Timestamp }

interface Directory {
    @tool(risk = R0, desc = "Lista o conteúdo de uma pasta")
    list(path: string<4096>) -> result<vec<DirEntry, 4096>, FsError>;

    @tool(risk = R2, desc = "Move um item para a lixeira")
    trash(path: string<4096>) -> result<(), FsError>;

    open(path: string<4096>, flags: OpenFlags) -> result<handle<File>,
FsError>;
}
```

Regras: todo tamanho é limitado na declaração; versões seguem semver; métodos novos só em versão menor; `handle<T>` transfere capability tipada.

14.4 Manifesto de serviço

```
[service]
name = "netd"
binary = "/sys/bin/netd"
priority = 150
restart = "always"

[capabilities]
needs = ["dev:net/*", "log:write", "time:read", "mem:quota:32MiB"]
provides = ["optimus.net.Socket@1", "optimus.net.Config@1"]

[limits]
max_threads = 16
ipc_rate = 200000
```

Manifestos são TOML no código-fonte, compilados para binário e assinados junto com a imagem.

14.5 Formatos de arquivo

Formato	Uso	Estrutura
ELF64	Kernel, serviços, drivers	Padrão; seções W^X; assinatura em seção `.optsig`

Formato	Uso	Estrutura
OBFS	bootfs	Seção 5.3
`.osi`	Imagem de sistema	OptFS somente leitura + raiz Merkle + manifesto assinado
`.opk`	Pacote de app	Componentes Wasm, manifesto de permissões, assets, assinatura do autor + assinatura da loja
`.omdl`	Pacote de modelo	GGUF + manifesto (arquitetura, quantização, licença, avaliações, RAM mínima)
`.oupd`	Atualização	Delta binário entre imagens + metadados TUF

14.6 Versões

Releases seguem o calendário `AAAA.N` (ex.: `2028.1`). A ABI de syscalls só muda em versão maior da `kabi`, com camada de compatibilidade por pelo menos dois releases.

15. Perfis de hardware e metas

Um único código-fonte gera cinco perfis por configuração de build (`--profile lite|basic|jarvis|premium|server`). As metas abaixo são critérios de aceite medidos no CI; RAM em repouso não inclui modelos de IA.

15.1 Hardware de referência

Perfil	CPU	RAM	GPU/NPU	Disco mínimo	Experiência
Lite	1 núcleo x86-64 ou ARM64	512 MiB	Não	4 GiB	Texto e comandos de voz simples
Básico	4 núcleos	8–16 GiB	Não	32 GiB	Conversa útil, latência de segundos
JARVIS	8+ núcleos	32 GiB	GPU 16 GiB ou NPU 40+ TOPS	256 GiB NVMe	Voz fluida, raciocínio, visão
Premium	16+ núcleos	64–128 GiB	GPU 24–48 GiB	1 TiB NVMe	Modelos 70B, vários agentes
Server	16–256 núcleos, NUMA	64 GiB–4 TiB	Opcional	Conforme carga	Headless, VMs, realms, SRE por IA

15.2 Metas por perfil

Métrica	Lite	Básico	JARVIS	Premium
RAM em repouso	⩽ 64 MiB	⩽ 150 MiB	⩽ 300 MiB	⩽ 300 MiB
Boot até a interface	⩽ 3 s	⩽ 5 s	⩽ 8 s	⩽ 8 s
Velocidade do planejador	⩾ 3 tokens/s	⩾ 8 tokens/s	⩾ 20 tokens/s	⩾ 15 tokens/s (70B)
Primeiro token	⩽ 3 s	⩽ 2 s	⩽ 400 ms	⩽ 600 ms
Voz ponta a ponta	⩽ 500 ms (comando)	⩽ 2 s	⩽ 800 ms	⩽ 800 ms
HUD	30 fps	30 fps	60 fps a 1440p	60 fps a 4K
Pico total com IA	⩽ 400 MiB	⩽ 8 GiB	⩽ 28 GiB	Conforme modelo

15.3 Metas comuns

- Kernel ⩽ 1 MiB; IPC ida e volta ⩽ 1 µs; troca de contexto ⩽ 2 µs; latência de IRQ até o driver ⩽ 5 µs.
- Imagem de sistema Lite ⩽ 256 MiB sem modelos; pacote de modelos Lite ⩽ 400 MiB.
- Server: eficiência de escala ⩾ 80% até 64 núcleos em IPC e rede; boot do kernel ⩽ 500 ms após o firmware.

15.4 Como o perfil é escolhido

No primeiro boot, o `svcmgr` mede CPU, RAM, GPU e disco e propõe o perfil. O usuário pode trocar, e o `oie` baixa ou ativa o pacote de modelos correspondente.

16. Repositório, toolchain e build

Um monorepo Git com um workspace Cargo e um orquestrador `cargo xtask`. Um único comando, `cargo xtask run`, compila tudo, monta a imagem e dá boot no QEMU.

16.1 Estrutura

```
optimus/
├── rust-toolchain.toml      # Rust estável fixado
├── Cargo.toml               # workspace
├── xtask/                   # build, imagem, QEMU, testes, assinatura
├── boot/optboot/
├── kernel/
│   ├── okernel/             # arch/{x86_64,aarch64}, mm, cap, ipc, sched,
irq, time, smp, syscall
│   └── kabi/                # ABI compartilhada kernel/usuário
├── lib/
│   ├── ostd/  orun/  optcrypt/  oidl-rt/  holo/  forge/
├── idl/                     # *.oidl
├── tools/
│   ├── oidlc/  mkimage/  mkobfs/  sign/
├── services/
│   ├── root/ svcmgr/ memd/ timed/ logd/ auditd/ crashd/ powerd/
│   ├── devmgr/ acpid/ storaged/ optfs/ netd/ tlsd/ wland/
│   ├── displayd/ inputd/ audiod/ authd/ keyd/ policyd/
│   ├── updated/ pkgd/ rescue/ cockpit/
│   └── drivers/{uart,virtio,pcie,iommu,nvme,ahci,ps2,hda,xhci,...}
├── ai/
│   ├── router/ planner/ reader/ exec/ memory/ voiced/ ctxd/ evolved/
│   ├── oie/                 # motor de inferência
│   └── ops/  opl/           # linguagens de plano e de política
├── apps/owr/                # runtime WebAssembly
├── tests/                   # integração e cenários em QEMU
├── fuzz/                    # alvos de fuzzing
├── bench/                   # benchmarks com metas da seção 15
└── docs/{sdd,adr,spec,threat-model}/
```

16.2 Toolchain

- Rust estável fixado em `rust-toolchain.toml`, atualizado a cada 6 semanas por PR dedicado. Nightly só em jobs de Miri e fuzzing.
- Alvos próprios: `x86_64-optimus-kernel`, `x86_64-optimus` (usuário), `aarch64-optimus-kernel`, `aarch64-optimus`, e `x86_64-unknown-uefi` para o `optboot`.
- Emulação: QEMU (q35 + OVMF; virt + AAVMF), com KVM quando disponível.
- Dependências externas só em `xtask`, `tools/` de host, testes, fuzzing e benchmarks, auditadas com `cargo-vet`.

16.3 Comandos `xtask`

Comando	Faz
`cargo xtask build --arch x86_64 --profile lite`	Compila tudo para o alvo
`cargo xtask image`	Gera ESP, bootfs, imagem de sistema e disco virtual
`cargo xtask run [--gdb] [--smp N] [--mem 512M]`	Boot no QEMU com serial no terminal
`cargo xtask test [unit|kernel|integration|all]`	Roda as suítes
`cargo xtask fuzz <alvo>`	Fuzzing local
`cargo xtask bench`	Mede e compara com as metas
`cargo xtask sign --key <id>`	Assina artefatos (chaves de teste no dev)

16.4 CI

- A cada PR: formatação, clippy, testes unitários, testes de kernel em QEMU (x86-64 e AArch64), integração, contagem de `unsafe`, tamanho do kernel, build reproduzível (dois builds, hash idêntico).
- Noturno: fuzzing contínuo (≥ 4 h por alvo crítico), Miri, Kani, benchmarks com gate de regressão de 5%, suíte de red team da IA.
- Release: builds em máquinas isoladas, assinatura com chaves intermediárias, publicação dos hashes.

17. Padrões de código e processo

O código do Optimus segue regras verificadas por máquina sempre que possível; o que não dá para verificar exige revisão humana dupla.

17.1 Rust

- Edition 2024; `rustfmt` padrão; `clippy` com `pedantic` e `-D warnings`.
- `#![forbid(unsafe_code)]` em toda crate, exceto as listadas em `unsafe-allowlist.toml` (kernel `arch/`, `mm/`, drivers, `ostd` baixo nível, kernels SIMD do `oie`).

- Todo bloco `unsafe` tem comentário `// SAFETY:` com as invariantes e é revisado por duas pessoas.
- Sem `panic!` em caminhos do kernel: erros são `Result` com tipos próprios; `panic = "abort"` em todo o sistema.
- Sem alocação em tratadores de interrupção e no caminho rápido de IPC.
- Tipos novos para unidades (`PhysAddr` , `VirtAddr` , `Pages` , `Nanos`) em vez de inteiros soltos.
- Funções públicas documentadas; crates com `#![deny(missing_docs)]` .

17.2 Fluxo de trabalho

- Branch curta por tarefa do backlog; PR pequeno (alvo ⩽ 400 linhas alteradas).
- Commits no padrão Conventional Commits com `Signed-off-by` (DCO).
- Mudança de arquitetura exige ADR nova aprovada antes do código.
- Mudança em TCB, `kabi` , cripto ou política exige revisão do responsável de segurança.

17.3 Definição de pronto

- [ ] Código revisado e aprovado
- [ ] Testes unitários e, quando couber, de kernel ou integração
- [ ] Alvo de fuzzing para todo parser ou interface que recebe dados de fora
- [ ] Orçamentos de tamanho, RAM e latência da seção 15 mantidos no CI
- [ ] Documentação e OIDL atualizados
- [ ] Sem novo `unsafe` fora da allowlist

17.4 Equipe e aprendizado

O projeto exige Rust de sistemas. Para quem vem de C#, TypeScript ou ABAP, a trilha recomendada é: *The Rust Programming Language*, *Rust for Rustaceans*, o tutorial *Writing an OS in Rust* e o livro *Operating Systems: Three Easy Pieces*, antes de tocar no kernel.

18. Testes e verificação

Quanto mais perto do kernel, mais forte a garantia: provas formais nas invariantes de capabilities e IPC, fuzzing em toda fronteira, testes comuns no resto. A IA tem sua própria suíte de avaliação e de ataque.

Nível	Ferramenta	Alvo	Gate
Especificação	TLA+	Protocolo de IPC, revogação, escalonador, atualização A/B	Modelo verificado antes do código do M2
Prova	Kani (model checking)	Derivação de capabilities, alocador buddy, tabelas de página	Provas no CI noturno
Unsafe	Miri	Crates da allowlist testáveis no host	Sem erros
Unitário	`cargo test` no host	Lógica portável do kernel e serviços	Cobertura ≥ 80% em kernel e `optcrypt`
Kernel	Harness próprio no QEMU	Syscalls, IPC, memória, SMP	Todos passam em x86-64 e AArch64
Fuzzing	libFuzzer no host + `okfuzz` (syscalls, estilo syzkaller)	Parsers, OIDL, syscalls, rede, ACPI, GGUF, Wasm	Nenhuma falha nova em 4 h
Integração	Cenários em QEMU com serial e capturas de tela	Boot, login, rede, atualização, rollback	100% verde
Desempenho	`bench/`	Metas da seção 15	Regressão ≤ 5%
Cripto	Wycheproof, NIST, dudect	`optcrypt`, `tlsd`	100% dos vetores; tempo constante
IA: qualidade	Suíte de tarefas próprias em pt-BR e en-US	Sucesso de tarefas, chamadas válidas, latência	Metas por perfil
IA: segurança	Red team de prompt injection (e-mail, web, arquivos, áudio)	Política e taint	Zero ações R3 sem confirmação; zero R4
Caos	Matar serviços e drivers aleatoriamente	Supervisão e recuperação	Sistema volta em ≤ 1 s

Toda falha encontrada vira teste de regressão antes da correção ser aceita.

19. Roadmap e marcos

São 16 marcos em quatro fases, cada fase fechada por um gate. A primeira versão que conversa e age, ainda em máquina virtual, sai no Gate 3; a v1.0 exige hardware real e as metas da seção 15.

A primeira versão que conversa e age chega no Gate 3, em VM



roadmap · 4 fases, 16 marcos, 4 gates

Dentro de uma fase, os marcos podem correr em paralelo quando a equipe permitir; um gate só passa com os testes de integração do bloco verdes e as metas de desempenho do componente atingidas. Não há datas fixas: o ritmo é medido pela velocidade real do M0 ao M2 e o plano é recalibrado no Gate 1.

20. Backlog inicial (M0 a M2)

As 33 tarefas abaixo levam do repositório vazio a processos isolados conversando por IPC no kernel próprio. A ordem é a de execução; cada item traz o critério de aceite.

20.1 M0: Fundação

- [ ] **T-001** Criar o monorepo com licença Apache-2.0, `SECURITY.md`, `CONTRIBUTING.md` (DCO) e `CODEOWNERS`. *Aceite:* repositório público ou privado configurado, proteção da branch principal.
- [ ] **T-002** `rust-toolchain.toml`, workspace Cargo e especificações de alvo `x86_64-optimus-kernel` e `x86_64-optimus`. *Aceite:* `cargo build` de uma crate vazia `no_std` para cada alvo.
- [ ] **T-003** `xtask` com `build`, `image` e `run` (QEMU q35 + OVMF, serial no terminal). *Aceite:* um comando abre o QEMU com o disco gerado.
- [ ] **T-004** CI com formatação, clippy, testes e smoke test de boot no QEMU. *Aceite:* PR com falha é bloqueado.
- [ ] **T-005** Converter as ADRs 0001–0024 em `docs/adr/`. *Aceite:* um arquivo por ADR com contexto, alternativas e consequências.
- [ ] **T-006** Publicar o modelo de ameaças da seção 13 em `docs/threat-model/`. *Aceite:* revisado e aprovado.
- [ ] **T-007** `optboot` mínimo: aplicação UEFI que escreve na tela e na serial. *Aceite:* `cargo xtask run` mostra "Optimus OS — optboot".
- [ ] **T-008** Iniciar especificações TLA+ de IPC e revogação. *Aceite:* modelo roda no TLC sem violar invariantes básicas.

20.2 M1: Kernel inicial (x86-64, um núcleo)

- [ ] **T-101** `optboot`: ler a ESP, validar e carregar o kernel ELF64 na metade alta. *Aceite:* segmentos carregados com permissões corretas.
- [ ] **T-102** Crate `kabi` com `BootInfo` v1; `optboot` preenche mapa de memória, GOP, RSDP e semente aleatória. *Aceite:* kernel imprime o mapa recebido.
- [ ] **T-103** Tabelas de página iniciais, KASLR, `ExitBootServices` e salto para `kernel_entry`. *Aceite:* endereço do kernel muda entre boots.

- [ ] **T-104** Entrada em assembly, pilha, zerar BSS, logger serial com níveis. *Aceite:* "okernel: hello" na serial.
- [ ] **T-105** GDT, TSS com pilhas IST, IDT e tratadores de exceção com dump de registradores. *Aceite:* falta proposital gera dump legível, não reboot silencioso.
- [ ] **T-106** Alocador buddy de frames. *Aceite:* testes no host com 10^6 operações aleatórias sem vazamento.
- [ ] **T-107** Gerência de tabelas de página, mapa direto, W^X, NX, SMEP, SMAP e UMIP. *Aceite:* teste de escrita em página de código falha como esperado.
- [ ] **T-108** Alocador slab e heap do kernel com limite. *Aceite:* estatísticas por cache no log.
- [ ] **T-109** x2APIC, calibragem do TSC, timer deadline e relógio monotônico. *Aceite:* desvio ⩽ 0,1% contra o HPET em 10 s.
- [ ] **T-110** Pânico: congela, registra e reinicia; mensagem também no framebuffer. *Aceite:* pânico de teste aparece na tela e na serial.
- [ ] **T-111** Harness de testes de kernel no QEMU com `isa-debug-exit`. *Aceite:* `cargo xtask test kernel` roda no CI.

20.3 M2: Capabilities, processos e IPC

- [ ] **T-201** `kabi`: números de syscall, erros, direitos e layout de mensagem (seção 14). *Aceite:* tipos compartilhados compilam no kernel e no usuário.
- [ ] **T-202** `MemoryQuota` e `obj_create`. *Aceite:* criação além da cota retorna `QuotaExceeded`.
- [ ] **T-203** CapSpace radix, handles com geração, `cap_mint`, `cap_delete`, `cap_info`. *Aceite:* handle reutilizado com geração antiga retorna `InvalidHandle`.
- [ ] **T-204** Árvore de derivação e `cap_revoke` preemptível. *Aceite:* prova Kani de que direitos nunca aumentam; revogação de 10^5 caps sem latência > 50 µs por fatia.
- [ ] **T-205** `AddressSpace`, `MemoryObject` e `vm_map` / `vm_unmap` / `vm_protect`. *Aceite:* mapear sem direito `MAP` falha.
- [ ] **T-206** `Thread`, troca de contexto com XSAVE, entrada de syscall segura (SYSCALL/SYSRET com checagens de endereço canônico). *Aceite:* duas threads de usuário alternando.
- [ ] **T-207** Escalonador de prioridade fixa com `SchedContext`. *Aceite:* thread sem orçamento não roda até o próximo período.
- [ ] **T-208** `Endpoint`, `Reply` e as quatro primitivas de IPC com prazo, mais o caminho rápido. *Aceite:* testes de ping-pong e de transferência de capability.
- [ ] **T-209** `Notification`, `obj_bind_notify`, `futex_wait` / `futex_wake`. *Aceite:* mutex de usuário com 4 threads sem perda de wake-up.

- [ ] **T-210** Faltas e exceções de usuário enviadas ao endpoint de exceções. *Aceite:* processo que acessa endereço nulo é encerrado e reportado.
- [ ] **T-211** Formato OBFS, ferramenta `mkobfs` e carregamento do `root`. *Aceite:* `root` inicia a partir do bootfs.
- [ ] **T-212** `ostd` mínimo e processo `hello` que conversa por IPC com o `root`. *Aceite:* troca de 10^6 mensagens sem erro.
- [ ] **T-213** Benchmarks de IPC e troca de contexto. *Aceite:* resultados publicados; meta ⩽ 1 µs e ⩽ 2 µs em KVM.
- [ ] **T-214** `okfuzz` v0, fuzzer de syscalls. *Aceite:* 4 h sem pânico do kernel.

Os backlogs dos marcos M3 em diante são detalhados ao fim de cada marco anterior, seguindo os critérios de saída da seção 19.

21. Riscos

Os três maiores riscos são o tamanho do escopo, os drivers de GPU e a segurança da IA agindo sozinha. Todos têm um plano B que preserva a entrega de valor.

Risco	Impacto	Mitigação	Plano B
Escopo grande demais para a equipe	Projeto não chega à v1.0	Marcos com entrega útil desde o M8 em VM; backlog só dos próximos marcos	Lançar "Optimus VM Edition" (só virtio) como v1.0
GPU do zero (Intel Xe) atrasa	Perfis JARVIS/Premium lentos	Inferência por CPU otimizada; GPU no M12	NPU ou modo híbrido com nuvem opcional
Prompt injection supera a política	IA faz ação indevida	Leitor isolado, taint, OPS, confirmação física, red team contínuo	Reduzir ferramentas R2/R3 disponíveis até corrigir
Cripto própria com falha	Quebra de confidencialidade	ADR 0004: código formalmente verificado, vetores, dudect, auditoria externa	Desativar algoritmo afetado; trocar para o par híbrido restante
Modelos locais fracos em hardware modesto	Experiência aquém do JARVIS	Roteador por gramática, skills, perfis honestos	Nuvem opcional com consentimento

Risco	Impacto	Mitigação	Plano B
Variedade de hardware real	Boot falha em máquinas comuns	Priorizar chipsets mais vendidos; relatórios de compatibilidade automáticos	Lista oficial de hardware certificado
Wi-Fi e firmware fechados	Sem rede sem fio	Catálogo de firmware assinado; Ethernet e USB primeiro	Adaptadores USB-Ethernet recomendados
Interpretador AML (ACPI) complexo	Falhas de energia e boot	Isolado em `acpid`, fuzzing com tabelas reais	Modo de energia mínimo
Falta de apps	Baixa utilidade diária	WebAssembly, IA gera ferramentas, camada POSIX	VM Linux para apps legados (M15)
Licenças de modelos mudarem	Distribuição bloqueada	ADR 0022, revisão a cada release	Trocar por modelo equivalente permissivo

22. Glossário

Termo	Significado
ADR	Registro de decisão de arquitetura
Badge	Valor de 64 bits gravado numa capability para identificar quem chama
Capability	Referência não falsificável a um objeto, com direitos; única forma de autoridade no Optimus
CapSpace	Tabela de capabilities de um processo
Confirmação física	Clique, toque ou tecla na Faixa de Confiança; voz não conta
Faixa de Confiança	Área da tela que só o compositor desenha (trusted path)
GGUF	Formato aberto de arquivo de modelos de linguagem

Termo	Significado
HUD Spec	Descrição declarativa de painéis que a IA envia ao Cockpit
IOMMU	Hardware que limita o acesso de dispositivos à memória (DMA)
KASLR	Endereço aleatório do kernel a cada boot
OIDL	Linguagem de definição de interfaces do Optimus
OPL	Linguagem de políticas do Optimus
OPS	Linguagem de planos da IA, limitada e verificável
Prompt injection	Texto malicioso em conteúdo externo que tenta comandar a IA
Realm	Grupo de processos com namespace, cotas e política comuns
R0–R4	Classes de risco de ações da IA
SchedContext	Orçamento de CPU (tempo por período) de uma thread
Taint	Etiqueta de origem de um dado (USER, SYSTEM, LOCAL, EXTERNAL, AI)
TCB	Base de confiança: o mínimo que precisa estar correto para o sistema ser seguro
TUF	Modelo de metadados de atualização resistente a comprometimento de chaves
W^X	Memória nunca é gravável e executável ao mesmo tempo